Otsu-Based Automatic Image Segmentation: Method, Implementation and Evaluation

M. Sai Charan

Roll Number: 24CS003246

Department of Computer Science / CSE AIML

Sri Padmavati? SPSU • Digital Image Processing Project • October 2026

Abstract — Image segmentation is an important task in digital image processing because it separates an image into meaningful regions. Manual threshold selection can be difficult when image brightness changes, so this project studies Otsu's thresholding method, which automatically selects a threshold from a grayscale image histogram. Otsu's method divides pixels into two classes—background and foreground—and chooses the threshold that maximizes between-class variance (equivalently, minimizes within-class variance). The proposed implementation reads an input image, converts it to grayscale, calculates the threshold, and produces a binary segmented image. A simple web interface can allow users to upload an image and compare the original, grayscale, and segmented outputs. The paper explains the mathematical basis, implementation workflow, expected observations, benefits, and limitations. Since no experimental image set or measured results were supplied, results are described qualitatively and should be replaced or supplemented with measurements from the completed implementation.

Keywords: Otsu thresholding, image segmentation, global thresholding, grayscale image, histogram, between-class variance, digital image processing.

1. Introduction

Image segmentation is the process of dividing an image into regions so that objects or important areas can be identified more easily. It is used in medical image analysis, document processing, object detection, industrial inspection, and many other computer vision tasks. A basic segmentation method is thresholding: each pixel is compared with a threshold value and assigned to one of two groups.

A fixed threshold must often be selected by the user and may not work well for every image. Otsu's method addresses this issue by calculating a threshold automatically from the image histogram. It evaluates possible thresholds and selects the one that gives the strongest separation between the foreground and background intensity classes. The method is simple, fast, and suitable for images whose histogram contains two reasonably distinct groups.

The main objectives of this project are: (i) understand automatic threshold selection; (ii) implement Otsu thresholding using Python and an image-processing library such as OpenCV; (iii) display the original and segmented images for comparison; and (iv) study the method's advantages and limitations.

2. Related Work

Thresholding has long been used as a straightforward approach to image segmentation. Global thresholding applies one threshold to the whole image, while local or adaptive thresholding calculates different thresholds for different regions. Nobuyuki Otsu introduced a widely used automatic threshold-selection technique in 1979. Rather than requiring the user to enter a threshold, the method uses the distribution of gray levels to select a value that separates two classes.

Later image-processing systems commonly included Otsu thresholding because it is computationally efficient and easy to reproduce. However, it is not a universal solution: images with uneven illumination, low contrast, or more than two overlapping intensity groups may not be segmented accurately by a single global threshold.

3. Methodology

3.1 Basic thresholding

For a grayscale pixel with intensity I(x,y) and threshold T, binary thresholding produces an output B(x,y):

B(x,y) = 255, if I(x,y) > T; otherwise **B(x,y) = 0.**

The value 255 represents white and 0 represents black in a standard 8-bit binary image. Depending on the image, the foreground and background may appear reversed; this does not change the core thresholding process.

3.2 Image histogram

A grayscale histogram counts how many pixels occur at each intensity level from 0 to 255. Otsu's method uses this distribution to test candidate thresholds. A histogram with two distinct peaks may indicate two groups of pixels, such as a dark background and a bright object.

3.3 Otsu's threshold selection

For each candidate threshold t, pixels are divided into class C0 (intensities at or below t) and class C1 (intensities above t). Let w0 and w1 be the proportions of pixels in the two classes, and let μ0 and μ1 be their mean intensities. The between-class variance is:

$$\sigma^2_B(t) = w0(t) \times w1(t) \times [\mu 0(t) - \mu 1(t)]^2$$

Otsu's method chooses the threshold t* that maximizes this variance:

$$t^* = \arg\max \sigma^2_B(t)$$

A large between-class variance means the two classes have well-separated mean intensities. The selected threshold is then applied to every pixel to create the binary segmented image.

3.4 Project workflow

Step	Operation	Purpose
1	Upload or read the input image	Get the image to process
2	Convert to grayscale	Use one intensity value per pixel
3	Calculate Otsu threshold	Automatically find a threshold
4	Apply binary thresholding	Separate pixels into two classes
5	Display and compare outputs	Review the segmentation result
6	Save/download output	Allow the result to be reused

4. Implementation

The project can be implemented in Python with OpenCV and a small HTML/CSS interface. OpenCV provides a built-in Otsu thresholding option, which calculates the threshold and returns the binary result. A typical processing sequence is to read the image, check that the file was loaded, convert a color image to grayscale, apply binary thresholding with Otsu's flag, and display or save the output.

Illustrative Python core:

```
import cv2

image = cv2.imread("image.jpg")
if image is None:
raise FileNotFoundError("Image not found")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
threshold_value, segmented = cv2.threshold(
gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Otsu threshold:", threshold_value)
cv2.imwrite("segmented_image.png", segmented)
```

In a web application, the frontend can provide an image-upload control and areas for the original image, grayscale image, and segmented output. A Python backend can perform processing and return the output image. The interface should also show the threshold value, provide an error message for invalid files, and allow the user to save the segmented image.

5. Experiments and Results

5.1 Experimental setup

To evaluate the project, test it on several images with different characteristics: (a) a dark object on a light background; (b) a light object on a dark background; (c) an image with a clear foreground and background; and (d) an image with shadows or uneven lighting. For each test, record the automatically selected threshold and inspect the resulting binary image.

5.2 Results and discussion

For an image with two clearly separated intensity groups, Otsu thresholding is expected to produce a useful separation without manual threshold tuning. The result can be checked visually by comparing the original image with the binary output. If the object and background have similar gray levels, or if illumination varies strongly across the image, the segmentation may include unwanted regions or miss parts of the object.

No actual input images or measured test results were included with the project brief. Therefore, this paper does not claim numerical accuracy, processing time, or a measured threshold for a particular image. After running the implementation, add a results table with the image name, Otsu threshold, visual observation, and any relevant metric available for your dataset.

Test image type	What to observe
Clear foreground/background	Whether the object is separated cleanly
Low-contrast image	Whether object and background merge
Uneven illumination	Whether one global threshold misses local details
Noisy image	Whether small speckles appear in the binary output

6. Advantages and Limitations

Advantages: automatic threshold selection; simple mathematical idea; fast processing; easy implementation; and no need to manually choose a threshold for each image.

Limitations: it uses one global threshold; performance can decrease with uneven lighting, noise, low contrast, or overlapping foreground/background intensities; and the standard method separates the image into two classes rather than identifying multiple semantic objects.

7. Conclusion

Otsu-based automatic image segmentation is a useful global thresholding technique for separating foreground and background in grayscale images. It selects a threshold by maximizing the between-class variance of the two pixel groups, reducing the need for manual tuning. The method is computationally simple and can be integrated into a Python image-processing program and a browser-based interface. Its quality depends on the image histogram, so testing on varied images is important. Future improvements may include noise reduction before thresholding, adaptive thresholding for uneven illumination, and quantitative evaluation on a larger dataset.

References

[1] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979. doi: 10.1109/TSMC.1979.4310076.

[2] R. C. Gonzalez and R. E. Woods, Digital Image Processing, Pearson. See the chapter on image segmentation and thresholding; verify the edition used.

[3] OpenCV Documentation, “Image Thresholding,” OpenCV documentation, https://docs.opencv.org/ (accessed October 2026).

[4] P. A. Liao, T. S. Chen, and P. C. Chung, “A Fast Algorithm for Multilevel Thresholding,” Journal of Information Science and Engineering, vol. 17, no. 5, pp. 713–727, 2001.